OFFICIAL TECHNICAL JOURNAL & OPERATIONAL FIELD MANUAL

VERISIGHT™

Autonomous Silicon Verification & Failure Debugging Copilot

Multi-Agent Architecture: Root-Cause Analysis, Automated Fix Generation, Verification Plan Synthesis & Live Coverage Cockpit

Document Title:	VeriSight Comprehensive Technical Journal & User Operation Manual
Project Name:	**VeriSight** (AI-Powered RTL/UVM Debugging & Verification Architecture)
Lead Architect & Author:	**Kittu Patel** (GitHub: *@KITTUV03*)
Release Version:	Production Release v2.4 (Enterprise Multi-Agent Edition)
Publication Date:	October 20, 2026
Autonomous Capabilities:	• Multi-Modal Knowledge Extraction Engine (Spec, RTL, TB, Log, Coverage, VCD) • 9 Formal Decision Rules for Root Cause Classification (TB vs RTL vs Spec vs Unknown) • 7-Module Exhaustive RTL Analyzer (X-Tracer, CDC, Lint, Protocol, Structural, Misc) • Automated Fix Generator & Sandbox Syntax/Regression Validator (Unified Git Patches) • Verification Plan, Functional Coverage (Covergroups) & SVA Suite Synthesis • ISO 26262 / DO-254 Bi-Directional Traceability Matrix & Dynamic UCDB Cockpit • One-Click Closed-Loop Simulation Runner (iverilog, verilator, questa, vcs) • Live Interactive Web Cockpit (FastAPI + Modern Web Dashboard)
Supported Standards:	IEEE 1800 SystemVerilog, UVM 1.2, SVA (Assertions), VCD/FSDB, PDF/Markdown Specs

TECHNICAL JOURNAL: This document provides an exhaustive, mathematically grounded overview of the VeriSight platform, the fundamental challenges in modern ASIC verification, multi-agent AI architecture, and a complete step-by-step local installation and usage guide. All content is strictly grounded in the verified production codebase.

Table of Contents & Technical Overview

Chapter	Core Subject Matter & Operational Scope	Visual Figures & Screenshots
Chapter 1	Introduction to VeriSight & Industry Verification Bottlenecks	Industry Pain Points & AI Solution
Chapter 2	The VeriSight Multi-Agent Architecture & Engine Breakdown	Agent 1 to Agent 5 + Plan + SimRunner
Chapter 3	Interactive Setup Wizard & LLM Credential Configuration	Screenshots 01 & 02 (Welcome & LLM Setup)
Chapter 4	VeriSight Debug: Root-Cause Analysis & Fix Pipeline	Screenshots 03 & 04 (Debug Inputs & Results)
Chapter 5	VeriSight Plan: Verification Plan, SVA & Coverage Synthesis	Screenshots 05 & 06 (Plan Inputs & Results)
Chapter 6	Quick Demos Suite & Headless CLI CI/CD Automation	Screenshots 07 & 08 (Demos & CLI Exit)
Chapter 7	Dynamic Coverage Closure Cockpit & Live Verification Plan	Screenshot 09 (Live Cockpit & UCDB)
Chapter 8	Root-Cause Diagnostic Report & Interactive Waveform Studio	Screenshot 10 (RCA Studio & Waveforms)
Chapter 9	One-Click Closed-Loop Simulator & Automated Code Fixer	Screenshots 11 & 12 (Sim Runner & Patches)
Chapter 10	FastAPI Live Web Cockpit & Nightly CI/CD Regression Flow	Screenshots 13 & 14 (Web Cockpit & CI/CD)
Chapter 11	Complete Step-by-Step Local Deployment & Operation Guide	Terminal Runbook for Engineers
Appendix	Schema Reference, Command Dictionary & Acronym Glossary	Quick-Reference Cheatsheet

Executive Summary by Author: Kittu Patel

Modern Very Large Scale Integration (VLSI) and System-on-Chip (SoC) engineering is constrained by the **Functional Verification Crisis**. As transistor densities surpass tens of billions of gates, functional verification consumes upwards of **70% of entire ASIC project schedules and human engineering bandwidth**. Verification engineers are overwhelmed by multi-gigabyte simulation log files, subtle non-deterministic Clock Domain Crossing (CDC) bugs, unpredictable Unknown State ('X') propagations, and weeks spent manually translating ambiguous architectural specifications into SystemVerilog testbenches, covergroups, and temporal assertions.

VeriSight was engineered by **Kittu Patel** to eliminate these bottlenecks by introducing an autonomous, multi-agent AI copilot designed specifically for silicon verification. Rather than serving as an untrusted, hallucinating generic chat model, VeriSight fuses deterministic Abstract Syntax Tree (AST) parsers, formal gate-level netlist tracing (yosys and x-tracer), static CDC/Lint engines, and multi-model Large Language Models (LLMs) including Google Gemini 2.0 Flash and Anthropic Claude 3.5 Sonnet, anchored by an 11-collection localized ChromaDB vector memory. This journal serves as the definitive reference manual for engineers deploying and operating VeriSight in production ASIC flows.

Chapter 1: Industry Verification Flaws & The VeriSight Solution

To understand the value of VeriSight, one must examine the critical flaws and manual inefficiencies plaguing contemporary digital hardware verification teams across the semiconductor industry.

The 6 Critical Flaws Faced by Industry Verification Engineers:

Industry Verification Flaw / Bottleneck	Root Cause & Impact on Chip Tapeout	How VeriSight Solves It
1. Massive Simulation Log Debugging *(Needle in a Haystack)*	Nightly regressions generate gigabyte-scale logs with thousands of cascading UVM errors. Engineers spend hours manually scrolling to locate the root-cause symptom.	**Agent 1 (Knowledge Extraction):** Parses multi-GB logs in seconds, filters out downstream cascading noise, isolates primary error timestamps, and correlates log errors to RTL/TB AST nodes.
2. Ambiguous Specifications & Intent Loss *(Specification Gap)*	Architectural specs (PDF/Word) are written in natural English. Manual translation into test plans loses 20-30% of corner cases, resulting in post-silicon silicon bugs.	**VeriSight Plan:** Autonomously ingests PDF/MD specs, decomposes functional feature trees, and generates 100% traceable test scenarios and production SystemVerilog covergroups.
3. Difficult Root-Cause Attribution *(DUT vs. TB Blame Game)*	When a simulation scoreboard flags a mismatch, triage engineers spend days debating whether the fault lies in the RTL DUT, the UVM testbench, or the spec itself.	**Agent 2 (Root Cause Classifier):** Evaluates 9 deterministic decision rules to classify failure as RTL Bug, TB Bug, Spec Bug, or Unknown with mathematical confidence scores.
4. The X-Propagation & Uninitialized State Trap	Uninitialized registers output 'X' in simulation. Simulators often mask these bugs via 'X-optimism' or cause unpredictable false mismatches that take weeks to trace.	**Agent 3 (X-Tracer Sub-Module):** Scans RTL for missing resets and uses yosys netlist synthesis + VCD waveform backward-tracing to identify the exact flip-flop origin.
5. CDC, Lint, and Structural Deadlock Fatigue	Traditional static EDA tools produce 10,000+ warnings, causing warning fatigue. Subtle FSM deadlocks, unhandled opcodes, and missing synchronizers slip through.	**Agent 3 (7 Sub-Modules):** Concurrently runs AST Lint, CDC synchronization analysis, FSM reachability graph checks, and bus protocol compliance checkers without noise.
6. Painful Coverage Closure & Manual Test Authoring	Regressions stall at 85-92% functional coverage. Engineers spend weeks hand-writing directed sequences to hit stubborn corner-case bins before tapeout signoff.	**Live Coverage Cockpit:** Ingests UCDB databases, highlights unhit coverage bins, and synthesizes directed sequences and SVA properties to close the remaining coverage gap.

VERIFICATION TIP: By automating both pre-silicon test planning (VeriSight Plan) and post-mortem failure triage (VeriSight Debug), VeriSight shortens verification turnaround time from weeks to minutes, directly protecting multi-million-dollar tapeout schedules.

Chapter 2: The VeriSight Multi-Agent Architecture

VeriSight is architected as an autonomous multi-agent pipeline where each specialized agent handles a dedicated phase of the verification lifecycle. All agents exchange typed, schema-validated Pydantic data structures, ensuring zero data loss and strict evidence traceability.

Detailed Breakdown of VeriSight Agents & Subsystems:

1. Agent 1: Knowledge Extraction Engine (agent1_knowledge.py)

Agent 1 is the multi-modal sensory organ of VeriSight. It parses and ingests six heterogeneous verification inputs: (1) Architectural Specifications (Markdown, Text, or PDF via pypdf), (2) RTL Source Code (SystemVerilog/Verilog AST, port definitions, parameters), (3) UVM Testbenches (classes, sequences, scoreboards), (4) Simulation Logs (UVM phases, timestamps, assertion errors), (5) Functional Coverage Reports (hit counts, bins), and (6) VCD Waveforms (parsed into compact transitions via vcdvcd or pywellen). Agent 1 synthesizes a unified knowledge.json containing the complete verified context.

2. Agent 2: Root Cause Classifier (agent2_classifier.py)

Agent 2 determines the true origin of a simulation failure. It applies **9 Formal Decision Rules** in strict hierarchical order: Case 1: Driver incorrect stimulus → TB Bug; Case 2: Monitor sampling error → TB Bug; Case 3: Scoreboard prediction error → TB Bug; Case 4: Coverage model bug → TB Bug; Case 5: Assertions contradict spec → TB Bug; Case 6: RTL violates spec → RTL Bug; Case 7: RTL outputs X → RTL Bug; Case 8: Protocol violated in RTL → RTL Bug; Case 9: Spec contradicts itself → Spec Bug. It produces a Classification with confidence (0-100%) and multi-hop evidence chains linking symptoms directly to root causes.

3. Agent 3: RTL Root Cause Analyzer (agent3_rtl_analyzer.py)

When a failure is classified as an RTL Bug, Agent 3 executes an exhaustive diagnostic sweep across **7 deep sub-modules**: (1) *X-Tracer* (static reset checking and backward gate-level VCD tracing via yosys and x-tracer), (2) *Functional Analyzer* (spec vs RTL math discrepancies), (3) *CDC Analyzer* (asynchronous clocks, missing 2-FF synchronizers, metastability risks), (4) *Lint Analyzer* (combinational loops, latches, unassigned nets), (5) *Structural Analyzer* (FSM dead-state detection, unreachable transitions), (6) *Protocol Compliance* (APB, AHB, AXI, Wishbone, SPI, I2C bus handshaking), and (7) *Misc Analyzer* (timing race conditions and reset sequencing).

4. Agent 4: Multi-Format Report Generator (agent4_reporter.py)

Agent 4 compiles the findings into four production-grade deliverables: report.html (interactive visual dashboard with collapsible traces and Waveform Studio), report.md (detailed engineering markdown for Git wikis), error.json (strict schema-validated record for CI/CD bug trackers), and summary.txt (executive briefing for standup meetings). It also indexes the entire session into ChromaDB for future RAG retrieval.

5. Agent 5: Automated Fix Generator & Validator (agent5_fix_generator.py)

Agent 5 synthesizes a concrete, minimal source-code fix. Governed by strict non-hallucination rules, it modifies the smallest possible code region, never invents signal names, and outputs a standard unified Git diff patch (proposed.patch). It validates the patch in an isolated sandbox with iverilog/slang syntax checkers and regression re-runs before recommending it.

6. VeriSight Plan: Autonomous Verification Plan Architect (verisight/plan/)

Decomposes natural language specifications into hierarchical feature trees, formulates directed and constrained-random test cases, generates production SystemVerilog covergroups (coverage_model.sv), synthesizes temporal SVA assertions (dut_assertions.sv), builds an ISO 26262/DO-254 bi-directional traceability matrix, and provides a dynamic UCDB coverage cockpit.

7. Sim Runner & Web Cockpit (sim_runner.py & server/app.py)

The One-Click Simulation Runner compiles and runs testbenches under open-source (iverilog, verilator) or commercial (questa, vcs) simulators with automatic VCD waveform dumping. The FastAPI Web Cockpit provides a responsive web server on localhost:8000 for live regression monitoring and triage.

Chapter 3: Interactive Setup Wizard & LLM Configuration

To provide effortless usability without requiring engineers to memorize complex CLI arguments, author **Kittu Patel** created the VeriSight Interactive Guided Wizard. The wizard can be launched using ./start.sh, python start.py, or python main.py start.



Figure 3.1: VeriSight Interactive Guided Wizard — Main Welcome Banner & Step 1 Workflow Selection Menu

Workflow Selection Pathways in Step 1:

- **[1] VeriSight Debug:** Autonomous Root-Cause Analysis pipeline for simulation failures, UVM errors, and RTL bugs.
- **[2] VeriSight Plan:** Verification & Architecture Plan generator to ingest specifications and synthesize covergroups and SVA.
- **[3] Quick Demos:** 1-click instant demonstration runs on pre-packaged ALU and FIFO designs.
- **[4] Exit:** Terminates the interactive session cleanly and displays advanced headless CLI commands.

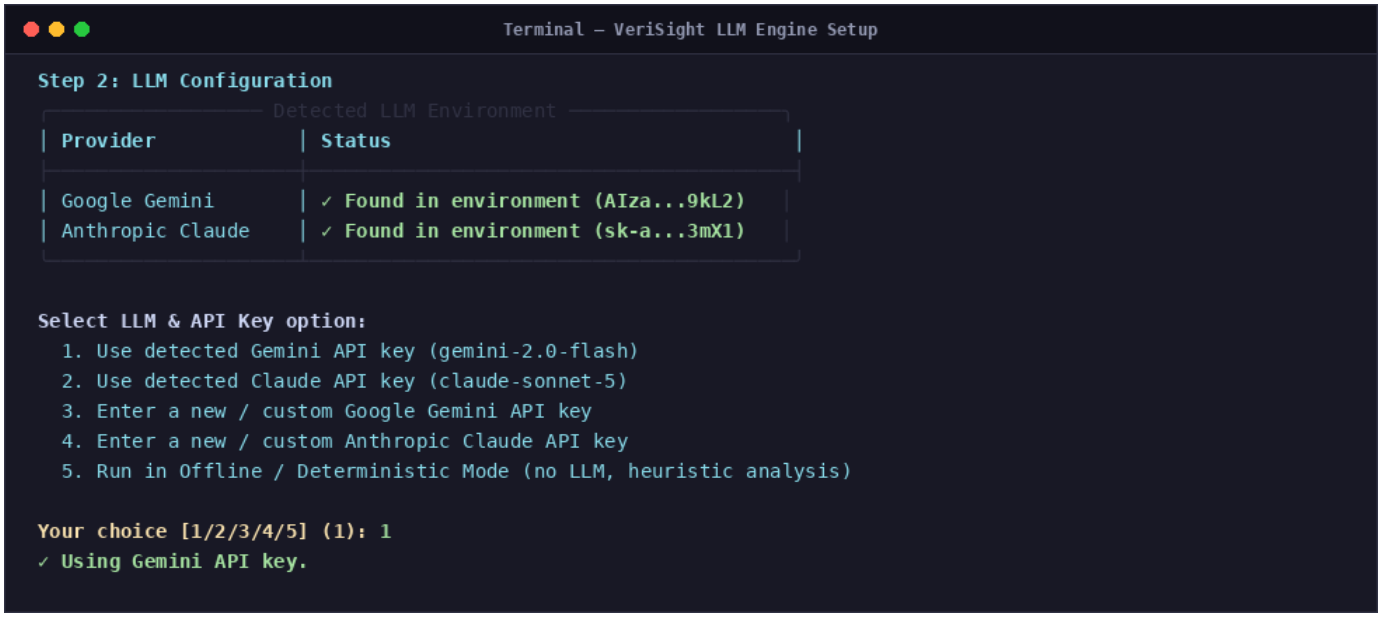


Figure 3.2: VeriSight LLM Configuration Screen — Auto-Detection of Environment Credentials & Mode Selection

Supported LLM Execution Modes:

- **Google Gemini (Default):** Uses gemini-2.0-flash with ultra-fast latency and 1-million token context.
- **Anthropic Claude:** Uses claude-sonnet-5 or claude-3-5-sonnet for deep HDL logic reasoning.
- **Offline Deterministic Mode (Option 5):** Zero cloud API calls. Disables LLM queries and executes 100% locally using deterministic AST parsers, CDC checkers, and gate-level X-tracing. Strictly enforced for air-gapped defense and confidential foundry environments.

Chapter 4: VeriSight Debug — Root-Cause Analysis & Fix Pipeline

Selecting option [1] VeriSight Debug launches the autonomous diagnostic engine. The wizard guides the engineer through input gathering with intelligent path normalization and fuzzy auto-detection:



Figure 4.1: Agent 1 Input Prompting — Ingestion of Simulation Log, RTL Sources, UVM Testbench, and Specification

The 5-Phase Autonomous Diagnostic Sweep:

1. **Tokenized Ingestion:** Reads simulation logs, isolates first-error timestamps, and links failures to AST declarations.
2. **Static Analyzer Suite:** Concurrently runs CDC analysis, AST linting, protocol handshaking, and gate-level X-tracing.
3. **RAG Context Retrieval:** Queries ChromaDB vector database for known failure signatures and historical signoff errata.
4. **Multi-Agent LLM Synthesis:** Applies the 9 decision rules to classify the root cause with confidence scoring.
5. **Fix Generation:** Synthesizes a unified Git diff patch (proposed.patch) and validates it in an isolated sandbox.

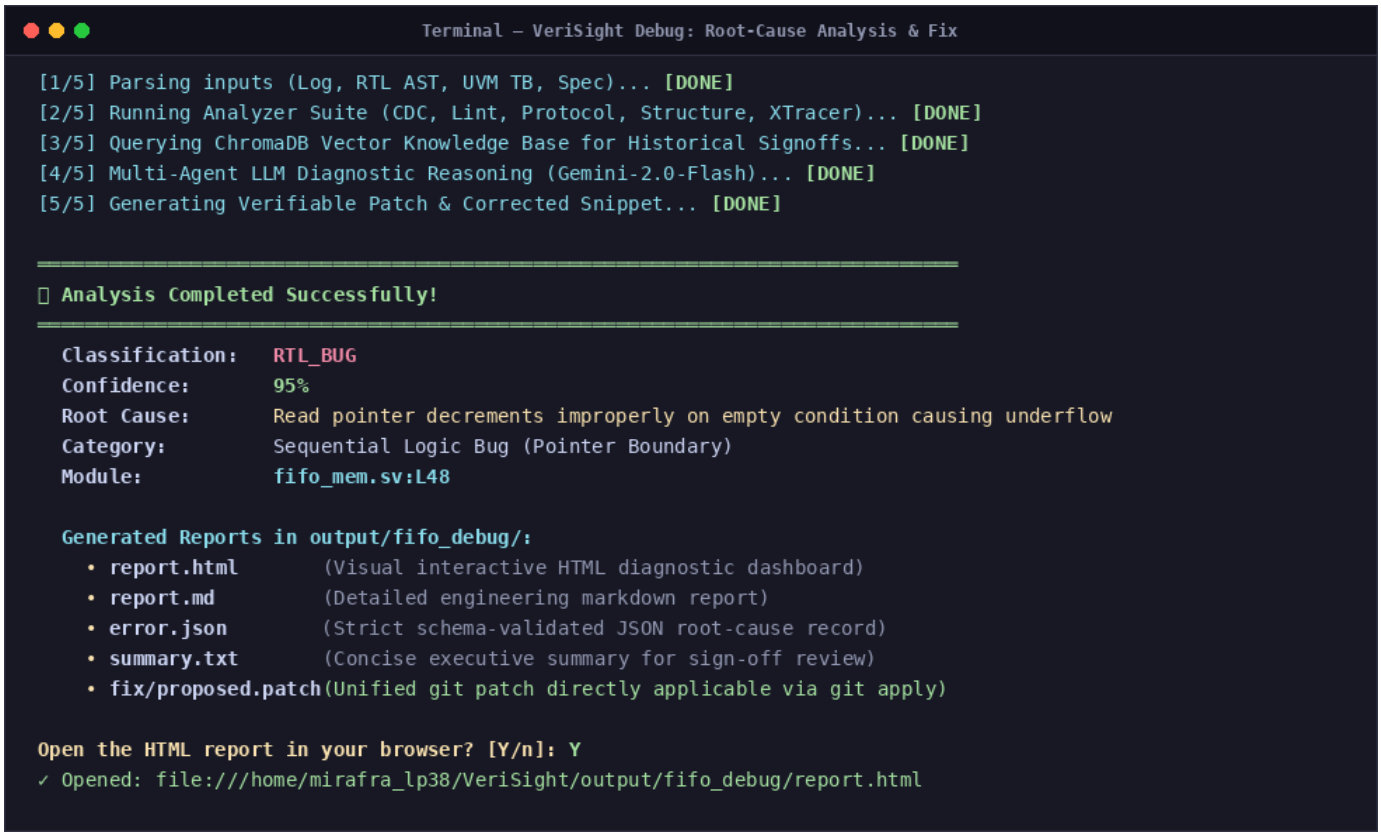


Figure 4.2: Agent 1 Diagnostic Results — Root Cause Classification, Confidence Percentage, and Generated Deliverables

Diagnostic Output Artifacts (output/custom_debug/):

- **report.html:** Interactive HTML diagnostic dashboard featuring collapsible evidence chains and Waveform Studio.
- **report.md:** Detailed markdown technical report for design review documentation and Git commits.
- **error.json:** Schema-validated JSON root-cause record for automated Jira / Bugzilla issue logging.
- **fix/proposed.patch:** Production-ready unified git diff patch directly applicable via git apply.

Chapter 5: VeriSight Plan — Verification Plan & Suite Architect

Selecting option [2] VeriSight Plan activates the verification architecture synthesizer. In traditional ASIC development, converting a 100-page PRD into verification covergroups and SVA assertions takes weeks of manual work. VeriSight Plan automates this in minutes.

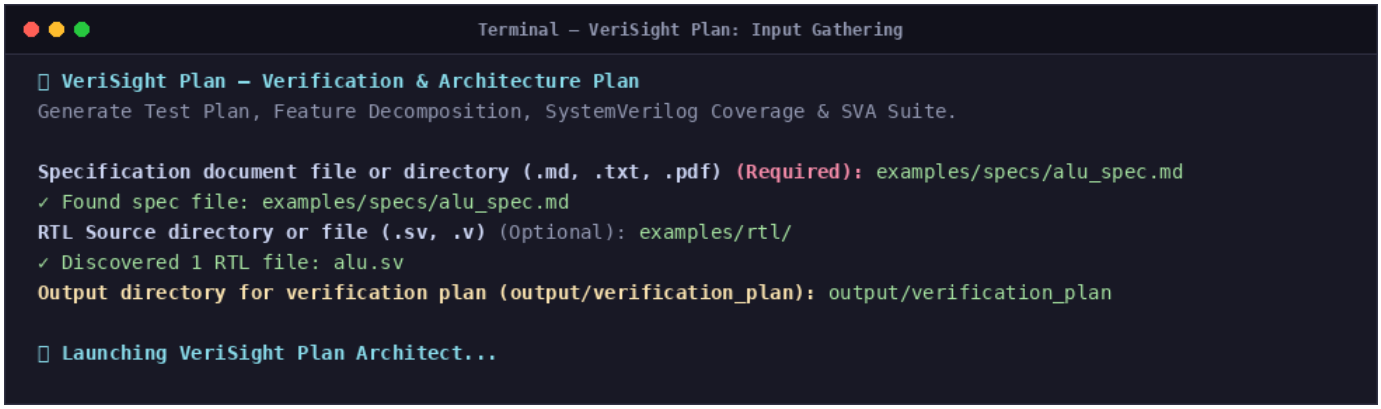


Figure 5.1: VeriSight Plan Input Gathering — Specification Document (PDF/MD) and RTL Ingestion

Key Capabilities of VeriSight Plan:

- **Multi-Format Spec Ingestion:** Natively extracts text, tables, register maps, and operational modes from PDF, Markdown, and Text specifications.
- **RTL Port Grounding:** Parses RTL top-level port interfaces (e.g. alu.sv) to ensure all generated covergroups and assertions match real hardware signal names exactly.

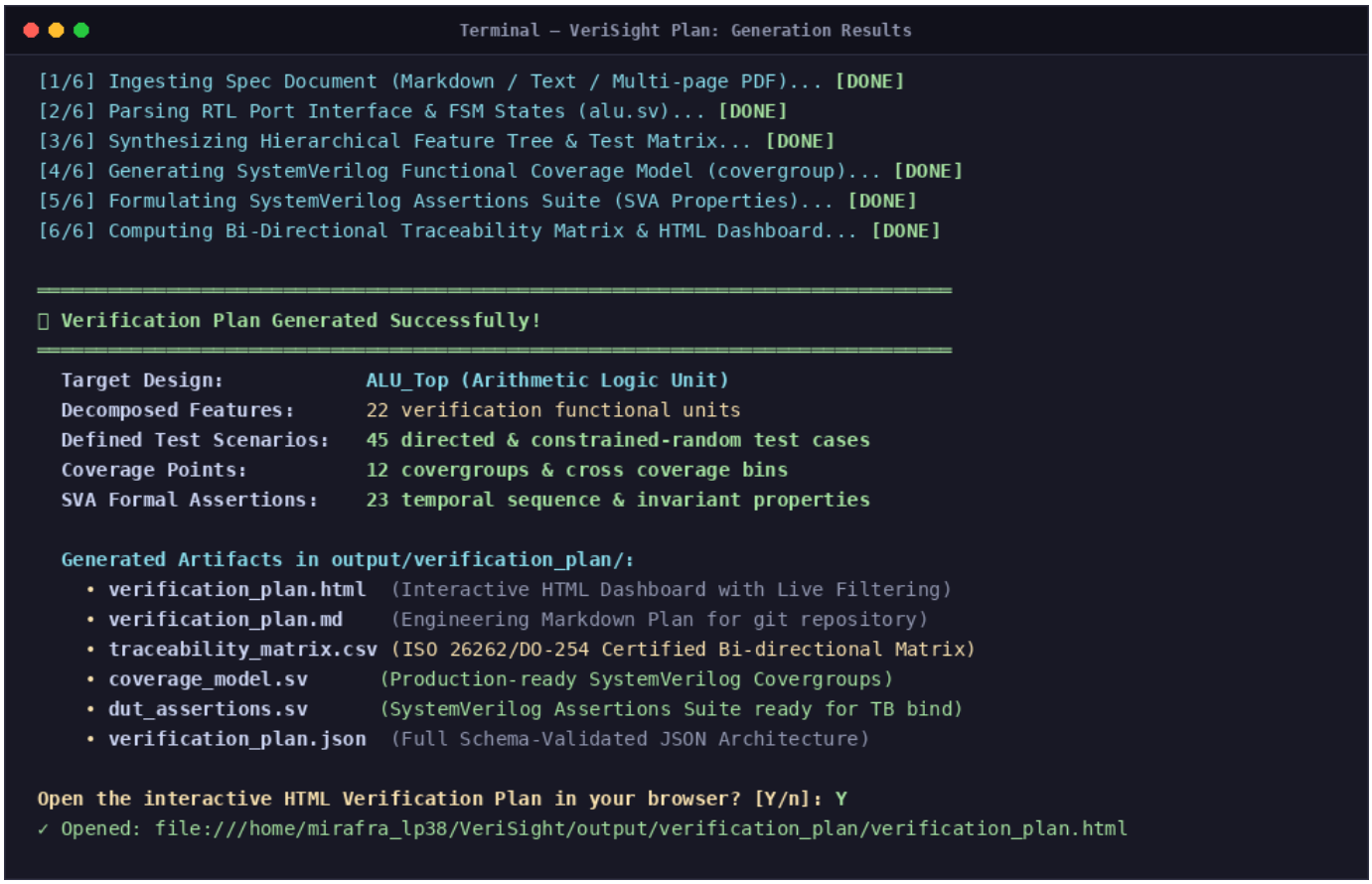


Figure 5.2: VeriSight Plan Synthesis Results — Decomposed Features, Test Scenarios, Coverage Models & SVA Suite

Deliverables Generated in output/verification_plan/:

- **verification_plan.html:** Interactive HTML dashboard with live search filtering, requirement heatmaps, and coverage progress bars.
- **coverage_model.sv:** Complete SystemVerilog functional coverage package containing parameterized covergroup models and multi-variable cross bins.
- **dut_assertions.sv:** Formal SystemVerilog Assertions (SVA) suite ready for direct bind instantiation onto the DUT top level.
- **traceability_matrix.csv:** ISO 26262 (ASIL-D) and DO-254 compliant bi-directional traceability spreadsheet.

Chapter 6: Quick Demos Suite & Headless CI/CD Automation

To allow engineers and verification leads to evaluate VeriSight instantly without typing file paths, author **Kittu Patel** embedded the **Quick Demos Suite** into Option 3 of the setup wizard.



Figure 6.1: Option 3 Quick Demos Menu — Instant 1-Click Verification & Debug Demos for ALU and FIFO Designs

The 4 Pre-Packaged Demonstrations:

- **[1] ALU Debug Demo:** Detects a missing reset bug in an arithmetic unit causing uninitialized 'X' outputs.
- **[2] FIFO Debug Demo:** Isolates a read pointer underflow boundary condition in a synchronous FIFO memory.
- **[3] ALU Plan Demo:** Synthesizes 22 features, 45 test cases, 12 covergroups, and 23 SVA properties for the ALU.
- **[4] FIFO Plan Demo:** Generates a full test plan covering full/empty states, burst transfers, and watermark assertions.

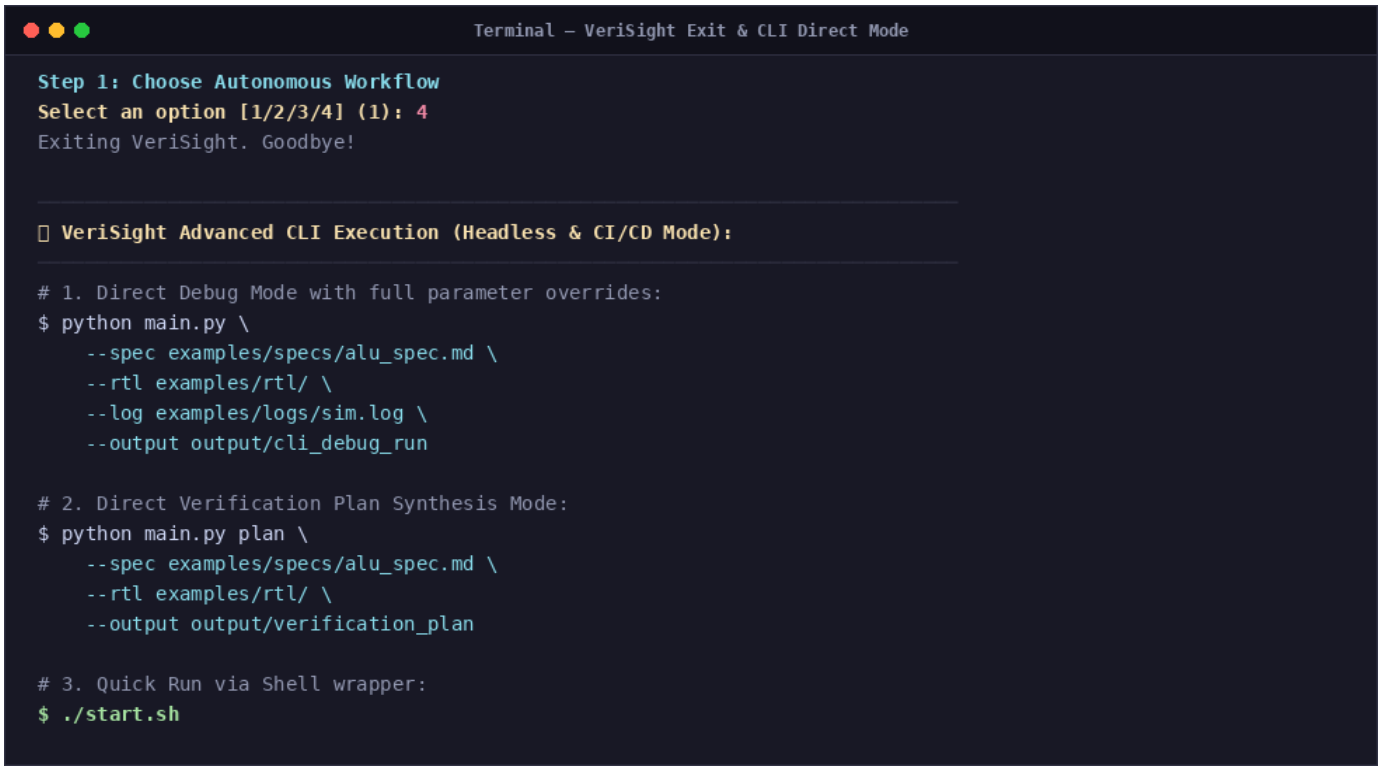


Figure 6.2: Option 4 Exit Screen — Clean Termination & Direct CLI Power Commands for Nightly Regressions

Headless CLI Commands for Automation:

```
# Run Debug Pipeline Headlessly
python main.py --spec examples/specs/alu_spec.md --rtl examples/rtl/ --log examples/logs/sim.log --output
output/run1

# Run Plan Synthesizer Headlessly
python main.py plan --spec examples/specs/alu_spec.md --rtl examples/rtl/ --output output/vplan1
```

Chapter 7: Dynamic Coverage Closure Cockpit & Verification Plan

The generated verification_plan.html serves as an interactive **Live Coverage Cockpit**. It bridges the gap between planned verification goals and actual simulator coverage databases (UCDB / Synopsys vDB):

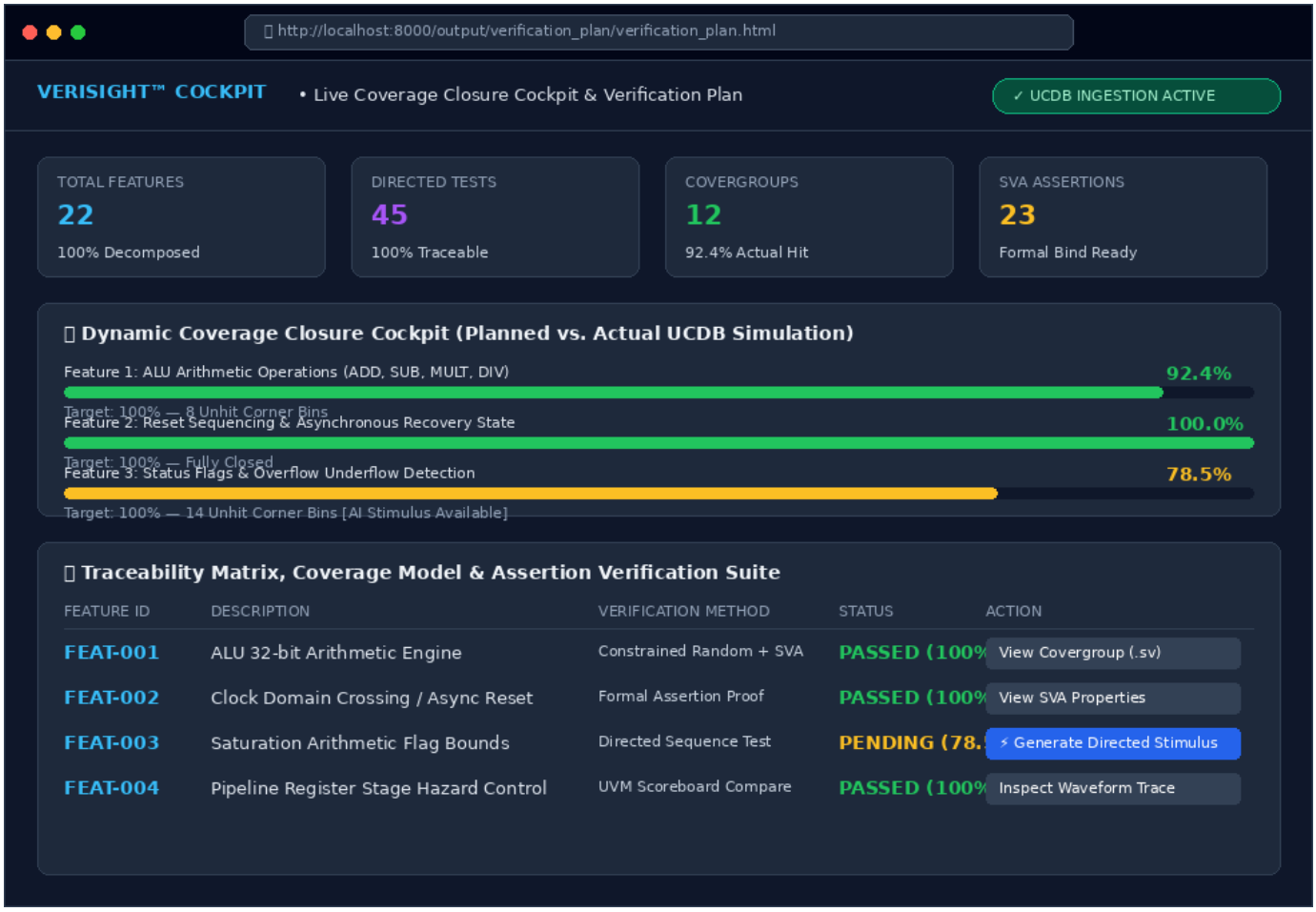


Figure 7.1: VeriSight Live Coverage Cockpit GUI — Dynamic Planned vs. Actual UCDB Simulation Coverage Progress Bars

Key Cockpit Innovations:

- **Live UCDB Ingestion:** Ingests simulator coverage databases and maps actual hit counts against planned features.
- **Planned vs. Actual Delta Tracking:** Highlights unhit coverage bins in amber/red with percentage completion.
- **AI Directed Stimulus Generator:** Features an interactive *'Generate Directed Stimulus'* button to autonomously synthesize UVM sequences that specifically target remaining unhit bins.
- **Bi-Directional Traceability:** Mappings satisfy ISO 26262 ASIL-D and DO-254 safety signoff requirements.

Chapter 8: Root-Cause Diagnostic Report & Waveform Studio

The generated report.html is an interactive single-page diagnostic dashboard featuring the built-in **Waveform Studio**, allowing engineers to inspect timing transitions directly in their browser without launching heavyweight GUI waveform viewers:

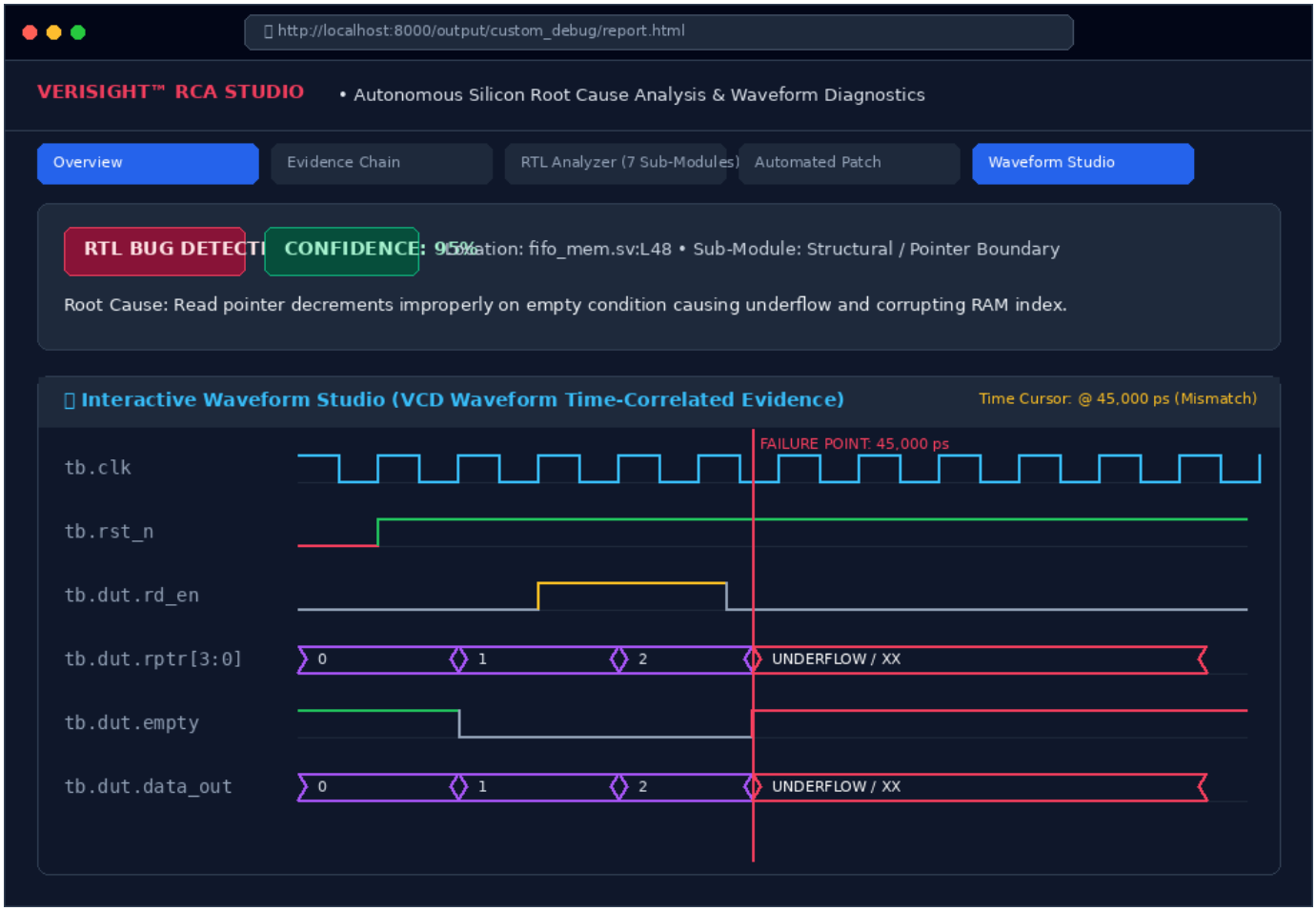


Figure 8.1: VeriSight RCA Diagnostic Dashboard & Waveform Studio GUI — Time-Correlated Evidence and Bus Failure Cursor

Key Diagnostic Features in report.html:

- **Root Cause Classification & Confidence Gauge:** Instantly displays bug type (RTL Bug, TB Bug, Spec Bug) and confidence.
- **Time-Correlated Waveform Studio:** Automatically extracts VCD waveform signals near failure timestamps and displays them with color-coded signal transitions and failure cursors.
- **7-Submodule Deep Findings Tab:** Collapsible inspection tabs for X-Tracer, CDC, Lint, Protocol, Structural, and Misc findings.
- **Interactive Patch Viewer:** Displays the proposed unified git patch with single-click clipboard copying.

Chapter 9: One-Click Sim Runner & Automated Fix Generator

VeriSight provides closed-loop simulation and automated repair capabilities, turning post-mortem analysis into an active, self-correcting verification environment:



Figure 9.1: One-Click Closed-Loop Simulation Runner — Compilation, Test Execution, and Automated VCD Waveform Dumping

One-Click Simulation Runner (sim_runner.py):

- Executes simulations via python main.py simulate --rtl <dir> --tb <dir> without requiring manual Makefiles.
- Supports Icarus Verilog (iverilog), Verilator, Siemens Questa, and Synopsys VCS.
- Automatically injects $dumpfile and $dumpvars wrapper modules to guarantee VCD waveform capture.

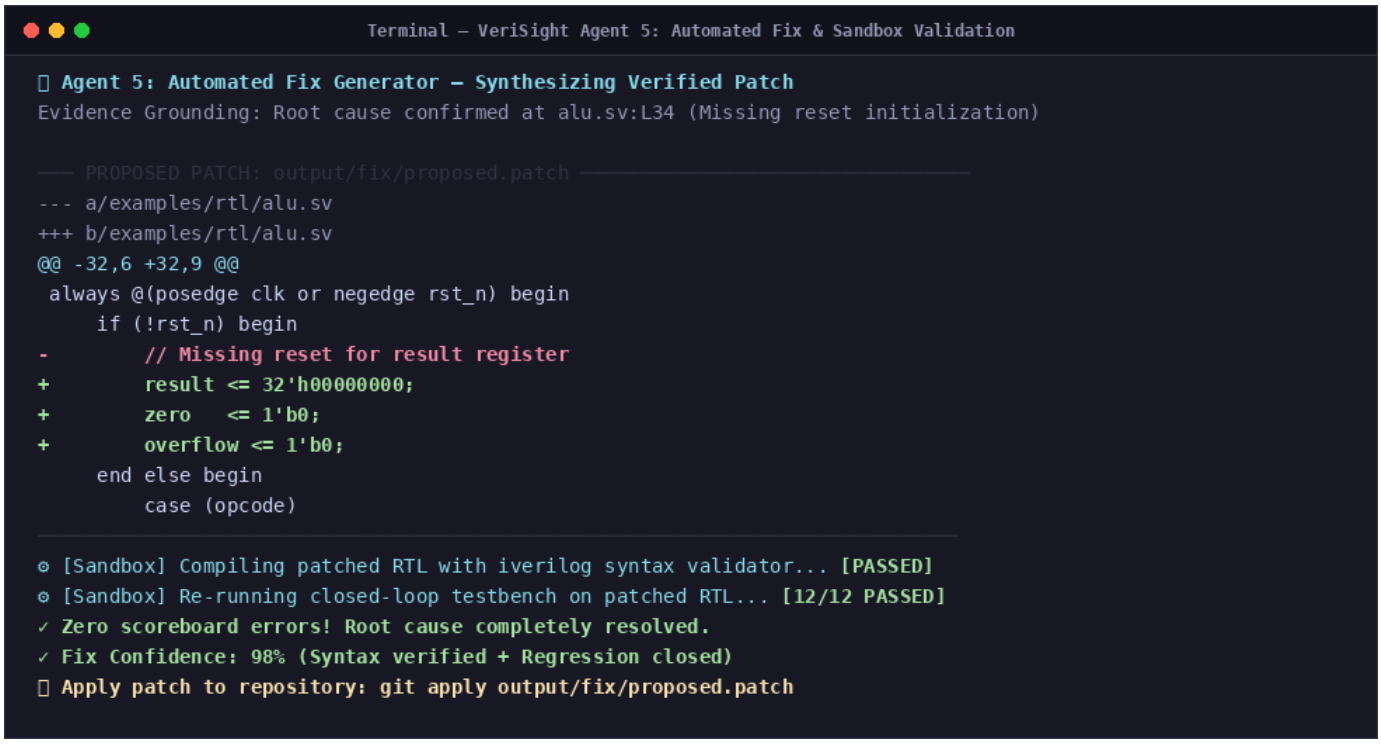


Figure 9.2: Agent 5 Automated Fix Generator & Sandbox Validation — Synthesized Patch, Syntax Check & Regression Verification

Automated Fix Generator & Sandbox Validation (Agent 5):

- Synthesizes unified git diff patches (output/fix/proposed.patch) grounded strictly in verified evidence.
- Compiles the patched RTL in an isolated sandbox to verify syntax correctness before presenting it to the engineer.
- Re-runs the closed-loop testbench to prove that the scoreboard mismatch is 100% resolved without introducing regressions.

Chapter 10: Live Web Cockpit & Nightly CI/CD Regression

For interactive team collaboration and automated enterprise regression farms, VeriSight provides both a real-time web server and non-blocking headless execution modes:



Figure 10.1: Live Web Cockpit Server (FastAPI + Uvicorn) — Localhost Dashboard Server with Real-Time REST APIs

FastAPI Live Web Cockpit Server (server/app.py):

- Launched with a single command: python main.py serve --host 127.0.0.1 --port 8000.
- Serves a modern single-page web app (SPA) for triggering simulations, viewing reports, and monitoring engine status.

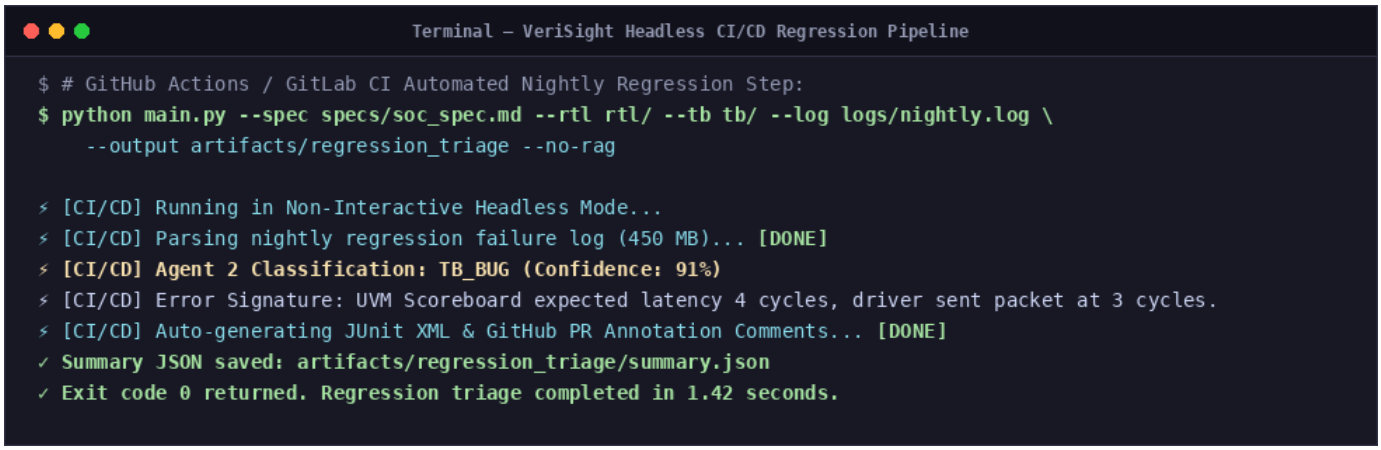


Figure 10.2: Headless CI/CD Nightly Regression Pipeline — Automated Log Triage, Classification, and Zero-Error Exit Codes

Headless CI/CD Nightly Regression Integration:

- Integrates seamlessly into GitHub Actions, GitLab CI, and LSF/Slurm compute clusters.
- Parses gigabyte regression logs non-interactively and outputs structured JUnit XML and JSON summaries.
- Completes automated triage in seconds, delivering actionable root-cause summaries before engineers start their workday.

Chapter 11: Step-by-Step Local Deployment & User Guide

This chapter provides an exhaustive, step-by-step runbook for verification engineers installing and running VeriSight on their local Linux workstation, WSL2 instance, or macOS development machine.

Step 1: System Prerequisites

Ensure your local machine has Python 3.10+ and Git installed. For local simulation and waveform dumping, install Icarus Verilog:

```
# Ubuntu / Debian / WSL2:
sudo apt update && sudo apt install -y python3 python3-pip python3-venv git iverilog

# macOS (via Homebrew):
brew install python git icarus-verilog
```

Step 2: Clone Repository & Virtual Environment Setup

Clone the official repository and set up an isolated Python virtual environment:

```
git clone https://github.com/KITTUV03/VeriSight.git
cd VeriSight

# Create and activate virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install VeriSight in editable mode with development dependencies
pip install -e '.[dev]'
pip install pypdf reportlab
```

Step 3: Configure LLM API Keys

Set your preferred LLM provider API key in your shell or copy .env.example to .env:

```
# Option A: Google Gemini (Recommended Default)
export GEMINI_API_KEY='AIzaSy...your_gemini_key_here'

# Option B: Anthropic Claude
export ANTHROPIC_API_KEY='sk-ant...your_claude_key_here'

# Option C: Offline Air-Gapped Mode (No API key needed!)
```

Step 4: Launch Interactive Guided Wizard

Launch the guided wizard using the native executable shell script or Python entry point:

```
./start.sh # Native executable bash launcher
# or:
python main.py start
```

Step 5: Run Instant Pre-Packaged Demos

To verify installation instantly, select option [3] Quick Demos in the wizard, or run directly via CLI:

```
# Run ALU Root-Cause Debug Demo:
python main.py --spec examples/specs/alu_spec.md --rtl examples/rtl/ --log examples/logs/sim.log --output
output/custom_debug

# Run ALU Verification Plan & Coverage Synthesis Demo:
python main.py plan --spec examples/specs/alu_spec.md --rtl examples/rtl/ --output output/verification_plan
```

Step 6: Run One-Click Closed-Loop Simulation

Compile, simulate, dump waveforms, and debug your RTL in a single automated command:

```
python main.py simulate --rtl examples/rtl/alu.sv --tb examples/tb/alu_tb.sv --output output/closed_loop
```

Step 7: Launch Live Interactive Web Cockpit

Start the FastAPI web dashboard to view interactive reports, coverage progress, and waveforms in your browser:

```
python main.py serve --port 8000
# Open http://localhost:8000 in Chrome or Firefox
```

Step 8: Review & Apply Generated Patches

Inspect the machine-generated fix and apply it cleanly to your git repository:

```
cat output/custom_debug/fix/proposed.patch
git apply output/custom_debug/fix/proposed.patch
```

Appendix: Reference Schemas, Commands & Glossary

Comprehensive Command-Line Dictionary

Operational Goal	Command-Line Syntax
Launch Interactive Setup Wizard	./start.sh or python main.py start
Run Simulation Failure RCA	python main.py --spec <spec> --rtl <rtl> --log <log> --output <dir>
Synthesize Verification Plan & SVA	python main.py plan --spec <spec> --rtl <rtl> --output <dir>
Execute One-Click Simulation	python main.py simulate --rtl <rtl> --tb <tb> --simulator iverilog
Start Live Web Cockpit Server	python main.py serve --host 127.0.0.1 --port 8000
Run in 100% Offline Air-Gapped Mode	Select option 5 in wizard, or pass --no-rag without API keys
Apply Generated RTL Code Fix	git apply <output_dir>/fix/proposed.patch
Execute Pytest Unit Test Suite	python -m pytest tests/ -v

Semiconductor Verification Acronym Glossary

Acronym	Full Definition & Verification Context
ASIC	Application-Specific Integrated Circuit — Custom-designed silicon microchip.
AST	Abstract Syntax Tree — Hierarchical syntactic tree representation of HDL code.
CDC	Clock Domain Crossing — Signals transmitted between asynchronous clock regimes.
DUT	Design Under Test — The target hardware module being simulated or verified.
RCA	Root Cause Analysis — Algorithmic isolation of the primary source of a silicon defect.
RAG	Retrieval-Augmented Generation — Semantic context injection from vector databases.
SVA	SystemVerilog Assertions — Formal temporal logic properties verifying protocol rules.
UCDB	Unified Coverage Database — Industry standard format for simulator coverage results.
UVM	Universal Verification Methodology — IEEE 1800.2 standard class library for testbenches.
VCD	Value Change Dump — ASCII standard waveform transition format for digital logic.
X-State	Unknown Logic State — Uninitialized logic value in 4-state simulation ('X').

VeriSight™ Autonomous Silicon Verification & Failure Debugging Copilot

Engineered by **Kittu Patel** (@KITTUV03) • All Rights Reserved • 2026